

Arquitectura de Modulos

PyME Financing Platform - Backend, frontend y estructura interna

1. Objetivo

Definir la organizacion interna del frontend, backend y paquetes compartidos para implementar el sistema de forma modular, mantenible y alineada al diseno de base de datos.

Decision arquitectonica

Usar un monolito modular: frontend Next.js, backend NestJS, Prisma/PostgreSQL y almacenamiento local de documentos. No usar microservicios en el MVP.

2. Estructura recomendada del repositorio

```
pyme-financing-platform/  
├── apps/  
│   ├── web/           # Frontend Next.js + TypeScript  
│   └── api/           # Backend NestJS + TypeScript  
├── packages/  
│   ├── database/      # Prisma schema, migraciones y seeders  
│   ├── uploads/       # Archivos locales en desarrollo  
│   └── docs/          # Documentacion tecnica en PDF  
├── docker-compose.yml  
├── .env.example  
└── README.md
```

3. Capas del backend

Capa	Responsabilidad	Ejemplo
Controller	Recibe HTTP, valida autenticacion/rol y delega al servicio.	CompaniesController, RiskAssessmentsController
DTO / Validation	Define forma de entrada y reglas de validacion.	CreateCompanyDto, UploadDocumentDto
Service	Contiene reglas de negocio y orquesta operaciones.	RiskAssessmentService.calculate
Repository / Prisma	Consulta y persiste datos usando Prisma.	prisma.company.findMany
Guard / Middleware	Protege rutas por JWT, rol y propiedad.	RolesGuard, OwnershipGuard
Interceptor / Audit	Registra acciones importantes de forma controlada.	AuditInterceptor

4. Modulos del backend

Modulo	Responsabilidad	Tablas principales	Dependencias
auth	Login, emision/validacion de JWT, hash de contraseñas.	users, roles	users
users	Gestion de usuarios activos e inactivos.	users, roles	roles, audit_logs
roles	Catalogo de roles internal_operator y applicant.	roles	seeders
companies	Registro, edicion y consulta de PyMEs.	companies	users, audit_logs
applications	Solicitudes de financiamiento y estados principales.	financing_applications	companies, status-history
document-requirements	Catalogo documental y reglas de aplicacion.	document_requirements	audit_logs
application-documents	Carga local, metadatos, revision y descarga segura.	application_documents	applications, users, audit_logs
financial-products	Catalogo de productos y elegibilidad minima.	financial_products	product-rules
product-rules	Scoring y compatibilidad por producto.	product_rules	financial-products
risk-assessments	Calculo de riesgo, snapshot e historial.	risk_assessments	applications, documents

Modulo	Responsabilidad	Tablas principales	Dependencias
application-matches	Recomendaciones por analisis de riesgo.	application_matches	risk-assessments, products, rules
status-history	Historial de cambios de estado.	application_status_history	applications, users
audit-logs	Bitacora interna de acciones relevantes.	audit_logs	users

5. Estructura interna del backend

```
apps/api/src/
  main.ts
  app.module.ts
  common/
    guards/roles.guard.ts
    guards/ownership.guard.ts
    decorators/current-user.decorator.ts
    filters/http-exception.filter.ts
    utils/money.ts
  modules/
    auth/
    companies/
    financing-applications/
    application-documents/
    risk-assessments/
    application-matches/
    financial-products/
    product-rules/
    audit-logs/
  prisma/prisma.service.ts
```

6. Estructura del frontend

Area	Responsabilidad	Ejemplo
routes/pages	Pantallas principales del sistema.	dashboard, companies, applications, products
components	Componentes reutilizables.	DataTable, StatusBadge, DocumentUploader
features	Logica por dominio de negocio.	companies, applications, risk, matching
services/api	Cliente HTTP tipado hacia backend.	companiesApi, documentsApi
auth	Sesion, token, usuario actual y rol.	AuthProvider, useCurrentUser
guards	Proteccion de rutas en frontend.	RequireRole, RequireAuth

```
apps/web/src/
  app/
    login/
    dashboard/
    companies/
    applications/
    financial-products/
    audit-logs/
  components/
  features/
  services/api/
  auth/
  lib/
```

7. Flujo de una solicitud HTTP

Paso	Descripcion
1	El frontend envia request con JWT en Authorization header.
2	AuthGuard valida token y usuario activo.
3	RolesGuard valida si el rol puede ejecutar la accion.
4	OwnershipGuard valida propiedad cuando el usuario es applicant.
5	DTO valida estructura y tipos de entrada.
6	Service ejecuta reglas de negocio.
7	Prisma guarda/consulta datos en PostgreSQL.
8	AuditLogService registra cambios relevantes si aplica.
9	Controller responde con formato estandar.

8. Dependencias criticas entre modulos

Dependencia	Razon
applications depende de companies	Toda solicitud debe pertenecer a una empresa existente.
application-documents depende de applications y document-requirements	Cada documento pertenece a una solicitud y a un tipo de requisito.
risk-assessments depende de applications y documents	El riesgo usa datos financieros, deuda y avance documental.
application-matches depende de risk-assessments	Las recomendaciones pertenecen al analisis que las genero.
product-rules depende de financial-products	Las reglas se agrupan por producto financiero.
audit-logs depende opcionalmente de users	Puede registrar acciones de usuario o procesos internos.

9. Convenciones de implementacion

Tema	Convencion
DTOs	CreateXDto, UpdateXDto, QueryXDto. No aceptar campos no definidos.
Servicios	Cada servicio debe tener metodos pequenos: create, findMany, findOne, update, deactivate.
Errores	Usar codigos estandar: VALIDATION_ERROR, FORBIDDEN, NOT_FOUND, INVALID_STATE_TRANSITION.
Montos	Usar Decimal/Prisma Decimal; no convertir a float para calculos financieros.
Archivos	Guardar en uploads/applications/{application_id}/ con stored_filename seguro.
Auditoria	Auditar cambios importantes, no auditar password_hash ni tokens.
Estados	Centralizar transiciones en ApplicationStatusService.

Anexo de Actualización

PyME Financing Platform

Documento base conservado: Arquitectura de Módulos

Este anexo no reemplaza ni resume el documento base. El contenido original se conserva íntegro. Las reglas de este anexo se agregan al diseño y prevalecen cuando una regla anterior sea ambigua o use lenguaje de sugerencia.

A. Decisiones confirmadas para todos los documentos

Decisión	Instrucción obligatoria
Registro de applicant	El applicant debe poder crear su propia cuenta desde una ruta pública usando, como mínimo, fullName, email y password. El sistema debe asignar el rol applicant automáticamente.
Resultado visible para applicant	El applicant no debe ver el análisis interno completo, risk_score, pesos de reglas, DSCR detallado, audit_logs ni comparación interna de productos. Solo debe ver la decisión final publicada por el operador.
Operación interna	El internal_operator debe ver risk_assessments, application_matches, razones internas, porcentajes, indicadores financieros y debe crear/publicar la decisión final.
Corrección de necesidad	El internal_operator debe poder validar o corregir la necesidad declarada por el applicant mediante validated_need_type antes de generar matching o decisión final.
Documentos rechazados	Cuando el applicant suba nuevamente un documento rechazado, el sistema debe reemplazar el archivo en el mismo application_document, actualizar metadatos, cambiar status a uploaded y registrar audit_log.
Auditoría	El sistema debe auditar solo acciones sensibles; no debe registrar absolutamente todos los cambios ni guardar secretos, tokens, password_hash o contenido de documentos.

B. Regla de redacción para implementación

Toda frase del documento base que use “recomendado”, “sugerido”, “podría” o “si se implementa” debe interpretarse como instrucción obligatoria cuando describa una regla del MVP, una validación de seguridad, una entidad de base de datos, un endpoint crítico, una transición de estado o una prueba mínima.

C. Módulos agregados o ajustados

Módulo	Responsabilidad obligatoria	Tablas
auth	Agregar registro público de applicant.	users, roles, audit_logs
financing-applications	Administrar need_type declarado y validated_need_type validado por operador.	financing_applications
application-decisions	Crear, editar antes de publicar, publicar y consultar decisiones finales.	application_decisions
application-matches	Generar opciones internas con porcentajes; no publicar automáticamente al applicant.	application_matches
application-documents	Reemplazar archivos rechazados en el mismo registro y auditar.	application_documents, audit_logs

Estructura backend agregada

```
apps/api/src/modules/application-decisions/ application-decisions.controller.ts
application-decisions.service.ts dto/create-application-decision.dto.ts
dto/publish-application-decision.dto.ts
```

Flujo HTTP actualizado

```
request -> AuthGuard -> RolesGuard -> OwnershipGuard -> DTO validation -> Service -> Prisma ->
AuditLogService -> response Para applicant, application-decisions debe filtrar por propiedad y
publicacion.
```